

FactCheck-Agent

Relazione tecnica su architettura backend e fine-tuning del classificatore NLI

Alessio Natale Scarpa
Alessio Pecilli

Progetto d'esame di Sistemi Intelligenti per Internet
Anno Accademico 2025–2026

Sommario

FactCheck-Agent è un sistema di verifica automatica di notizie che combina recupero di evidenze dal Web, classificazione logica del claim e generazione di una spiegazione argomentata. Il progetto integra un frontend *Next.js*, un livello *Backend-for-Frontend* per validazione e proxying delle richieste e un backend *FastAPI* orchestrato tramite *LangGraph*. Il contributo scientifico principale consiste nel fine-tuning di un classificatore *Natural Language Inference* basato su `cross-encoder/nli-deberta-v3-base` sul dataset `pietrolesci/nli-fever`, con ricerca degli iperparametri su learning rate, weight decay ed epoche.

La migliore configurazione sperimentale raggiunge sul validation set un'accuracy del 74.3% e un macro-F1 del 74.1%, con un miglioramento marcato rispetto alla baseline non fine-tunata. Dal punto di vista ingegneristico, il backend attuale implementa una pipeline a quattro nodi: raffinamento automatico della query, retrieval tramite DuckDuckGo, classificazione NLI con checkpoint DeBERTa fine-tuned locale e generazione della motivazione tramite modello generativo remoto. In questa architettura il verdetto fattuale non è affidato all'LLM, che viene utilizzato esclusivamente per supportare la ricerca e produrre una spiegazione testuale interpretabile.

1 Introduzione

La verifica automatica di affermazioni testuali è un problema centrale nei sistemi intelligenti per il Web, soprattutto in scenari caratterizzati da forte eterogeneità delle fonti, rapida obsolescenza dell'informazione e alta pressione temporale nella produzione del giudizio. In questo contesto, una pipeline di *fact-checking* credibile non può ridursi alla sola generazione di una risposta plausibile: deve invece recuperare evidenze pertinenti, valutarne la coerenza con il claim e restituire un output strutturato e ispezionabile.

Il progetto **FactCheck-Agent**¹ affronta tale esigenza con una soluzione full-stack che unisce componenti di retrieval, orchestrazione agentica e inferenza NLI. L'utente inserisce una notizia o un claim in linguaggio naturale; il sistema reperisce materiale correlato dal Web, elabora un verdetto tra **SUPPORTS**, **REFUTES** e **NOT ENOUGH INFO**, e produce una spiegazione in italiano accompagnata dal contesto recuperato.

Dal punto di vista progettuale, il lavoro ha due obiettivi distinti ma complementari. Il primo è ingegneristico: realizzare un'applicazione web modulare con contratti API tipizzati, validazione multilivello e backend facilmente estendibile. Il secondo è modellistico: addestrare un classificatore NLI specializzato sul dominio FEVER, valutandone in modo quantitativo l'impatto rispetto al modello pre-addestrato di partenza. La relazione si concentra quindi sugli aspetti realmente caratterizzanti del progetto: architettura backend, pipeline di fine-tuning e interpretazione critica dei risultati.

¹<https://github.com/Alessio-Pecilli/sii-2026>. Il repository contiene codice sorgente, notebook, artefatti sperimentali e il checkpoint fine-tuned del classificatore in `backend/fever-nli-deberta`, oltre ai file `experiment_results.csv`, `hf_test_predictions.csv` e alle figure usate in questa relazione.

Tabella 1: Stack tecnologico per layer applicativo

Layer	Tecnologie e ruolo
Frontend	Next.js 16.1.6, React 19.2.3, TypeScript, Tailwind CSS 4, lucide-react per UI e visualizzazione del report
BFF	Next.js Route Handler per validazione client-facing e proxy verso backend Python
Backend	FastAPI, Pydantic v2, uvicorn, endpoint <code>/verify</code> , inizializzazione modelli in fase di startup
Orchestrazione	LangGraph con quattro nodi operativi: <code>refine_query</code> , <code>search_web</code> , <code>nli_classification</code> , <code>generate_motivation</code>
Retrieval	DuckDuckGoSearchRun da langchain-community
LLM generativo	ChatGoogleGenerativeAI con modello <code>gemma-4-31b-it</code> , usato per affinamento query e motivazione
Classificatore NLI	checkpoint locale fine-tuned in <code>backend/fever-nli-deberta</code> ; fallback a <code>cross-encoder/nli-deberta-v3-large</code>

2 Architettura del Sistema e Backend

2.1 Vista d'insieme

L'applicazione segue un'architettura a tre livelli con pattern *Backend-for-Frontend* (BFF). Il frontend, sviluppato in *Next.js 16* con *React 19* e *TypeScript*, gestisce input utente, upload di file `.txt`, visualizzazione dello stato della pipeline e rendering del report finale. Il Route Handler `/api/verify` costituisce il livello BFF: valida il payload, normalizza la richiesta e inoltra il contenuto al backend Python. Il backend, implementato in *FastAPI*, espone l'endpoint REST POST `/verify` e invoca il grafo LangGraph che esegue la verifica.

La Figura 1 rappresenta la pipeline logica del sistema ed è coerente con l'implementazione corrente del backend. La separazione tra affinamento della query, ricerca, classificazione logica e spiegazione finale consente di distinguere il momento decisionale supervisionato dal momento generativo esplicativo, riducendo il rischio di attribuire al modello generativo il ruolo di arbitro fattuale.

2.2 Backend FastAPI e contratti API

Il backend definisce schemi Pydantic fortemente tipizzati: `VerifyRequest` accetta una `query` con lunghezza compresa tra 10 e 2000 caratteri; `VerifyResponse` restituisce `verdict`, `confidence`, `explanation`, `sources`, `nli_results`, `search_queries` e `retrieved_docs`. Tale struttura è replicata nel frontend tramite tipi TypeScript omologhi, riducendo il rischio di mismatch tra livelli applicativi.

L'applicazione inizializza i modelli nel *lifespan* di FastAPI: il modello generativo remoto e il classificatore DeBERTa locale vengono caricati in fase di startup, mentre il grafo LangGraph viene compilato una sola volta e riutilizzato per le richieste successive. Se il workflow non è ancora disponibile, l'endpoint POST `/verify` restituisce HTTP 503.

2.3 Grafo LangGraph e logica di esecuzione

Nel codice attualmente deployato il grafo LangGraph è composto da quattro nodi. Il primo, `refine_query`, utilizza il modello generativo per sintetizzare una query di ricerca breve e focalizzata. Il secondo, `search_web`, interroga DuckDuckGo con tale query e memorizza l'output in `researches`. Il terzo, `nli_classification`, confronta il claim utente con il testo recuperato usando il checkpoint DeBERTa fine-tuned locale, calcola i logits, applica una softmax e produce sia la classe sia la confidenza associata. Il quarto, `generate_motivation`, costruisce una spiegazione discorsiva in italiano a partire dal verdetto NLI e dalle evidenze estratte. Il flusso è quindi:

START $\rightarrow$ `refine_query` $\rightarrow$ `search_web` $\rightarrow$ `nli_classification` $\rightarrow$ `generate_motivation` $\rightarrow$ END.

Una parte ingegneristicamente rilevante è la netta separazione dei ruoli tra modelli. Il classificatore supervisionato decide il verdetto e la relativa confidenza; il modello generativo non modifica la decisione, ma

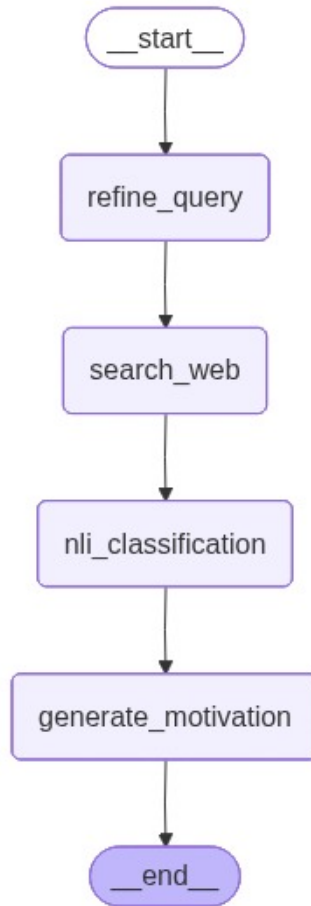


Figura 1: Pipeline del backend: raffinamento query, retrieval, classificazione NLI e generazione della motivazione

contribuisce unicamente alla formulazione della query iniziale e alla spiegazione finale. Di conseguenza il campo `nli_results` dell'API deriva da una vera inferenza del modello DeBERTa fine-tuned, mentre il testo di `explanation` rappresenta uno strato XAI discorsivo costruito sopra il risultato supervisionato.

2.4 Ruolo del frontend e del livello BFF

Il frontend non si limita a inviare la richiesta, ma svolge tre funzioni precise. In primo luogo, offre una validazione immediata sul claim e gestisce l'acquisizione di testo sia da textarea sia da file locali. In secondo luogo, il BFF centralizza le chiamate al backend Python e traduce errori di rete o di disponibilità in messaggi utente coerenti, ad esempio nel caso di backend non raggiungibile. In terzo luogo, il pannello risultati espone non solo il verdetto finale ma anche numero di documenti recuperati e contenuto di `nli_results`, favorendo una forma di trasparenza operativa utile in un'applicazione di fact-checking.

Lo *stepper* visuale della UI presenta i passaggi *Ricerca*, *NLI* e *LLM* come fasi distinte. Nel backend questa separazione è effettivamente presente, anche se il primo stadio reale include un raffinamento automatico della query non esplicitato graficamente. Un limite residuo dell'implementazione corrente è invece il campo `sources`: la risposta FastAPI espone il campo per coerenza di contratto, ma l'implementazione attuale lo lascia vuoto e concentra la trasparenza sui documenti grezzi recuperati.

Un ulteriore limite metodologico riguarda la valutazione del retrieval. Nel progetto non sono state raccolte metriche quantitative come Recall@K, Precision@K o *evidence hit rate* sui documenti restituiti da DuckDuckGoSearchRun; di conseguenza non è possibile affermare in modo misurato quante volte i documenti recuperati contengano effettivamente le evidenze necessarie per supportare o confutare il claim. L'analisi del retrieval resta quindi qualitativa: il sistema espone gli snippet recuperati, ma la loro sufficienza informativa non è stata benchmarkata contro un gold standard di evidenze.

Dal punto di vista della latenza, il repository non contiene ancora benchmark end-to-end o misure distribuzionali del tempo di risposta. È però possibile osservare che la pipeline attuale serializza due dipendenze

Tabella 2: Split utilizzati nella pipeline di addestramento

Split	Campioni	Label disponibili	Utilizzo
Train	5000	Sì	Fine-tuning del classificatore
Validation	1000	Sì	Selezione degli iperparametri e analisi comparativa
Test	1000	No (-1)	Generazione predizioni finali in <code>hf_test_predictions.csv</code>

esterne, retrieval Web e chiamata al modello generativo remoto `gemma-4-31b-it`, mentre la classificazione NLI viene eseguita localmente. Di conseguenza la latenza percepita dall’utente non dipende più dal tempo di un LLM per emettere il verdetto, ma soprattutto dal costo di rete del retrieval e della generazione della spiegazione; la decisione supervisionata resta invece, in prospettiva, il componente più prevedibile e potenzialmente più facile da ottimizzare lato serving.

3 Processo di Fine-Tuning

3.1 Dataset, obiettivo e preprocessing

Il classificatore supervisionato è addestrato nel notebook `notebooks/fever_nli_finetuning.ipynb` sul dataset Hugging Face `pietrolesci/nli_fever`, che fornisce coppie *premise/hypothesis* già allineate al task di *Natural Language Inference*. Le etichette utilizzate sono direttamente mappabili sul problema di fact-checking: `SUPPORTS` per entailment, `NOT ENOUGH INFO` per neutral e `REFUTES` per contradiction.

Per contenere il costo computazionale, il notebook lavora su sottoinsiemi controllati degli split originali, riportati in Tabella 2. La tokenizzazione è di tipo cross-encoder: premessa e ipotesi vengono fornite congiuntamente al modello con padding a lunghezza massima e troncamento a `max_length = 256`. Questa scelta è coerente con un task di classificazione della relazione logica tra due testi, in cui l’attenzione incrociata tra le due sequenze è più importante dell’efficienza di retrieval tipica dei bi-encoder.

3.2 Configurazione del training

Il modello di partenza è `cross-encoder/nli-deberta-v3-base`, scelto come compromesso tra capacità rappresentativa e sostenibilità dell’addestramento su GPU T4 in ambiente Colab. La procedura di ottimizzazione esplora sei configurazioni effettive, variando *learning rate*, *weight decay* ed epoche. Nel training sono inoltre applicate tecniche esplicite di controllo della memoria: *gradient checkpointing*, *gradient accumulation* pari a 2, batch size fisico di 4 esempi e batch di valutazione di 8 esempi.

Le metriche registrate sul validation set sono accuracy, precision, recall, macro-F1 e validation loss. L’uso del macro-F1 come criterio di selezione del modello migliore è appropriato, poiché il task è tri-classe e la qualità globale del classificatore non può essere riassunta in modo affidabile dalla sola accuracy.

Gli output salvati nel notebook permettono anche di stimare il costo temporale del training su GPU T4. Le configurazioni a 2 epoche richiedono circa 8–12 minuti per run: 11:34 per `LR=1e-5`, `WD=0.01`, 11:51 per `LR=2e-5`, `WD=0.01`, 10:25 per la configurazione migliore `LR=3e-5`, `WD=0.1` e 08:52 per `LR=5e-5`, `WD=0.01`. Le configurazioni a 3 epoche completano invece il training in 13:22 (`LR=2e-5`, `WD=0.01`) e 14:21 (`LR=5e-6`, `WD=0.1`). Pur trattandosi di tempi raccolti nel contesto specifico di Colab, essi mostrano che l’intera grid search resta economicamente sostenibile su hardware didattico, mentre un retraining sistematico su dataset più ampi richiederebbe una pianificazione computazionale più rigorosa.

4 Analisi dei Risultati e Discussione

4.1 Risultati della grid search

La Tabella 3 mostra una tendenza chiara. Un learning rate eccessivamente basso produce una convergenza insufficiente: la configurazione con 5×10^{-6} si ferma a un macro-F1 di 0.628, nettamente inferiore alle altre. All’estremo opposto, il learning rate più alto testato (5×10^{-5}) migliora la velocità di apprendimento

Tabella 3: Risultati sperimentali sul validation set da `experiment_results.csv`

Modello	LR	WD	Epoche	Accuracy	Macro-F1
Model_LR_3e-05_WD_0.1	3×10^{-5}	0.1	2	0.743	0.741
Model_LR_2e-05_WD_0.01	2×10^{-5}	0.01	3	0.737	0.734
Model_LR_2e-05_WD_0.01	2×10^{-5}	0.01	2	0.736	0.733
Model_LR_5e-05_WD_0.01	5×10^{-5}	0.01	2	0.726	0.722
Model_LR_1e-05_WD_0.01	1×10^{-5}	0.01	2	0.713	0.704
Model_LR_5e-06_WD_0.1	5×10^{-6}	0.1	3	0.645	0.628

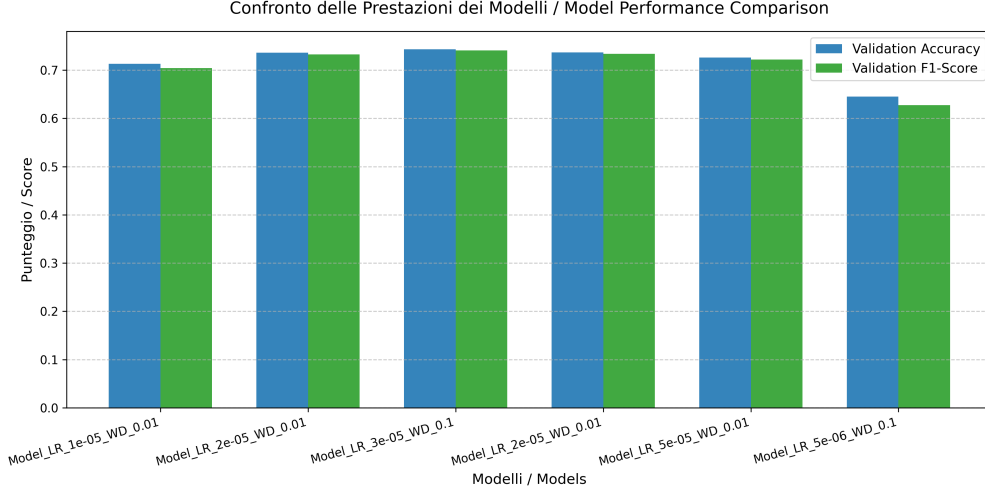


Figura 2: Confronto tra accuracy e macro-F1 delle configurazioni esplorate

ma non offre il miglior compromesso finale. La combinazione migliore risulta invece quella con 3×10^{-5} e weight decay 0.1, che raggiunge il miglior equilibrio tra accuratezza e generalizzazione.

L'incremento da 2 a 3 epoche sulla configurazione con 2×10^{-5} produce un guadagno trascurabile: accuracy da 0.736 a 0.737 e macro-F1 da 0.733 a 0.734. In termini ingegneristici, questo suggerisce che l'ottimizzazione utile si esaurisce molto rapidamente sul sottoinsieme di dati adottato e che l'aumento del tempo di training oltre le due epoche, in tale configurazione, non è giustificato da un beneficio apprezzabile.

4.2 Impatto del fine-tuning

Il dato più significativo dell'intera pipeline è il confronto riportato in Tabella 4. La baseline non fine-tunata, pur essendo già pre-addestrata su compiti NLI, mostra sul validation set una qualità del tutto insufficiente per il dominio specifico del progetto. Il fine-tuning aumenta l'accuracy di 56 punti percentuali assoluti e quasi quintuplica il macro-F1. In altre parole, il valore del progetto non risiede soltanto nell'integrazione applicativa, ma soprattutto nell'adattamento supervisionato del classificatore al dominio FEVER.

Questo risultato ha anche una lettura architettonica: il backend integra effettivamente il checkpoint fine-tuned nella pipeline online, rendendo la fase di decisione molto meno dipendente dal comportamento non deterministico del modello generativo e spostando l'LLM sul solo piano esplicativo.

4.3 Interpretazione qualitativa dei grafici

Il grafico in Figura 2 conferma visivamente la compattezza del gruppo di modelli migliori e il degrado netto delle configurazioni con learning rate troppo basso. La Figura 3 aggiunge due informazioni utili. In primo luogo, le curve di training e validation loss mostrano che le configurazioni migliori convergono entro poche epoche, senza evidenze marcate di overfitting sul set di sviluppo adottato. In secondo luogo, la matrice di confusione indica che la classe più problematica è NOT ENOUGH INFO, come atteso in un task in cui l'assenza di evidenze è semanticamente più ambigua del supporto o della contraddizione esplicita.

Tabella 4: Confronto tra baseline pre-addestrata e miglior modello fine-tuned sul validation set

Modello	Accuracy	Precision	Recall	Macro-F1
Base non fine-tunato	0.1830	0.2472	0.1810	0.1493
Miglior modello fine-tuned	0.7430	0.7500	0.7438	0.7406

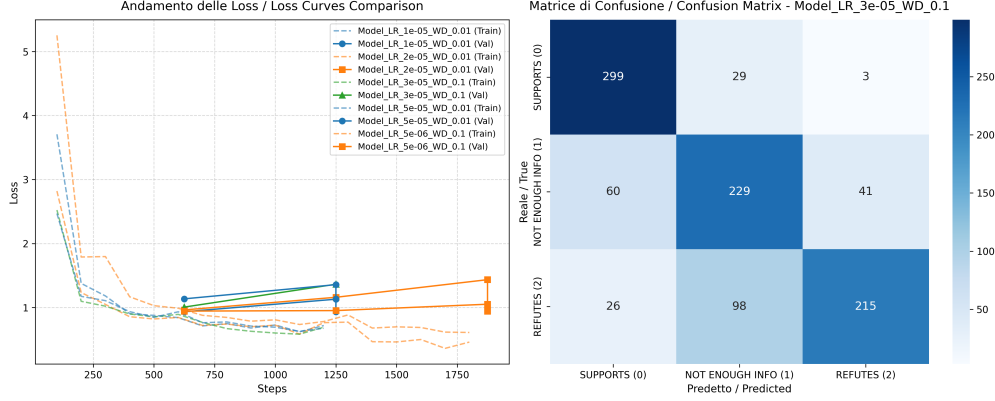


Figura 3: Curve di loss train/validation e matrice di confusione del modello migliore

Va inoltre precisato che il file `hf_test_predictions.csv` contiene predizioni sul test split ufficiale, le cui etichette nel dataset Hugging Face sono impostate a -1. Di conseguenza non è possibile stimare metriche quantitative su tale insieme: il file ha valore come output di inferenza finale o come base per analisi qualitative, ma non per una validazione comparativa oggettiva.

5 Pro, Contro e Sviluppi Futuri

Il progetto presenta diversi punti di forza. Sul piano software, l'architettura è modulare, tipizzata e facilmente estendibile; il BFF isola il frontend dai dettagli del backend Python; il backend carica in startup sia il classificatore locale sia il modello generativo remoto e li coordina in una pipeline chiara. Sul piano modellistico, il fine-tuning è documentato, riproducibile e produce un miglioramento quantitativo molto netto rispetto alla baseline. Sul piano dell'usabilità, l'interfaccia restituisce non solo un verdetto ma anche documenti recuperati e livello di confidenza.

I limiti principali si spostano ora su altri aspetti. Il primo riguarda il retrieval: DuckDuckGo fornisce snippet testuali grezzi, utili per un prototipo ma meno controllati delle evidenze curate in FEVER, e la loro qualità non è stata misurata con metriche standard. Il secondo riguarda la spiegazione: il testo generato dal modello remoto è coerente con il verdetto supervisionato, ma resta comunque un passaggio generativo non garantito da una struttura citazionale formale. Il terzo limite è infrastrutturale: il sistema non include persistenza, rate limiting, logging strutturato o calibrazione formale della confidence. Un quarto limite applicativo è che il campo `sources` dell'API non è ancora popolato automaticamente.

Le evoluzioni più naturali sono quindi tre. La prima è il miglioramento del retrieval mediante re-ranking semantico o selezione di fonti più affidabili. La seconda è l'arricchimento dell'output con fonti esplicite e con una spiegazione più strutturata, ad esempio con citazioni ancorate alle evidenze. La terza è la trasformazione dell'architettura in un deployment più esplicito, ad esempio separando frontend, API backend e model server, così da mantenere distinto il servizio di inferenza classificatoria dal servizio generativo.

6 Conclusione

FactCheck-Agent dimostra che una pipeline di fact-checking ben progettata richiede sia una buona architettura software, sia un componente di inferenza specializzato e valutato in modo rigoroso. Il repository contiene entrambe le dimensioni: una web app completa, un backend agentico a quattro nodi e un percorso di fine-tuning supervisionato con risultati convincenti. La parte più forte del progetto è il classificatore NLI

fine-tuned, che porta il modello da prestazioni quasi inutilizzabili a un macro-F1 di circa 0.74 sul validation set e che partecipa direttamente alla decisione online.

Dal punto di vista ingegneristico, il backend attuale rappresenta un prototipo credibile perché separa il livello decisionale supervisionato dal livello esplicativo generativo. La conclusione più importante è quindi che il progetto non si limita più a proporre l'integrazione futura tra ricerca e sistema operativo: tale integrazione è già presente nella pipeline corrente. I margini di miglioramento riguardano soprattutto la qualità del retrieval, la strutturazione delle fonti e la maturazione infrastrutturale del servizio.

Riferimenti bibliografici

- [1] J. Thorne, A. Vlachos, C. Christodoulopoulos e A. Mittal. FEVER: a large-scale dataset for Fact Extraction and VERification. In *Proceedings of NAACL-HLT*, 2018.
- [2] P. He, X. Liu, J. Gao e W. Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations*, 2021.
- [3] T. Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, 2020.
- [4] S. Ramírez. FastAPI documentation. <https://fastapi.tiangolo.com/>
- [5] LangChain Inc. LangGraph documentation. <https://langchain-ai.github.io/langgraph/>